UNIVERSIDAD NACIONAL DE LUJÁN

Departamento de Ciencias Básicas • Lic. en Sistemas

Programación Orientada a Objetos

(11[0|2]76)

Clase 07 • **Ejercicio 04** • Ciclo Lectivo 2026

MATERIAL PARA ESTUDIANTES
ENUNCIADO PRÁCTICO

Ejercicio 4: Plataforma de Streaming y Procesamiento de Contenidos

Contexto

En este ejercicio combinaremos por primera vez una **Clase Abstracta** (que modela una familia taxonómica "es-Un" con identidad y estado compartido) con una **Interfaz de Capacidad** (que modela un rol transversal "puede-Hacer" / "se-comporta-como").

Además, aplicaremos el **Principio de Inversión de Dependencias (DIP)**: los componentes de alto nivel deben depender de abstracciones (interfaces de rol) y no de implementaciones concretas, permitiendo procesar objetos heterogéneos de manera desacoplada y extensiva.

Enunciado

Una plataforma de streaming multimedia requiere rediseñar su subsistema de gestión y procesamiento de catálogo.

En la plataforma conviven diferentes tipos de contenidos que derivan de la clase abstracta `ContenidoMultimedia`:

- **Atributos protegidos:** `titulo: String`, `duracionSegundos: int` y `creador: String`.

- **Constructor protegido:** Inicializa los atributos y valida que `titulo` y `creador` no sean nulos ni vacíos, y que `duracionSegundos` sea estrictamente mayor a 0 (de lo contrario, arroja `IllegalArgumentException`).
- **Método abstracto:** `public abstract String reproducir();`
- **Métodos accesores públicos:** `getTitulo()`, `getDuracionSegundos()` y `getCreador()`.

Las clases concretas que especializan este catálogo son:

1. **Canción (`Cancion`):** Posee el atributo `calidadKbps: int` (ej. 320 kbps). Su método `reproducir()` retorna un mensaje descriptivo indicando el título, artista y tasa de bits de decodificación.
2. **Película (`Pelicula`):** Posee el atributo `resolucion: String` (ej. "1080p", "4K"). Su método `reproducir()` retorna un mensaje descriptivo indicando el título, director y resolución de proyección.
3. **Podcast (`Podcast`):** Posee los atributos `nroEpisodio: int` y `temporada: int`. Su método `reproducir()` retorna un mensaje descriptivo indicando el título, anfitrión, número de episodio y temporada.

Adicionalmente, se requiere dotar a ciertos contenidos de la capacidad de almacenamiento local para su consumo fuera de línea:

- **Interfaz de Capacidad `Descargable`:** Declara las operaciones esenciales `void descargar()` y `boolean isDescargado()`.
- **Regla de negocio de la plataforma:** Por políticas de licenciamiento de las productoras, la plataforma autoriza el almacenamiento local de música y largometrajes (administrando internamente el estado booleano `descargado`, inicialmente `false` y modificado a `true` al descargarse). Por el contrario, los podcasts son emisiones bajo demanda que deben transmitirse exclusivamente vía streaming en línea y no admiten descarga fuera de línea. Analice la jerarquía y aplique la interfaz `Descargable` de manera selectiva según corresponda.

Por último, se debe diseñar la clase orquestadora `GestorDeDescargas`:

- Administra una colección `List<Descargable> colaDescargas`.
- `encolarDescarga(Descargable d): void`: agrega un elemento descargable a la cola (validar que no sea nulo).

- `procesarDescargas(): int` : ejecuta secuencialmente el método `descargar()` sobre cada elemento encolado, vacía la cola y retorna el total de descargas procesadas con éxito.

Tareas:

1. Diseñe la clase abstracta `ContenidoMultimedia` con sus atributos protegidos, constructor con validación de invariantes, accesores y método abstracto `reproducir()`.
2. Diseñe la interfaz de capacidad `Descargable` con los métodos `descargar()` e `isDescargado()`.
3. Implemente las subclases concretas (`Cancion` , `Pelicula` , `Podcast`) especializando su comportamiento y asignando selectivamente la interfaz `Descargable` a aquellas clases que admitan almacenamiento local según el dominio.
4. Implemente la clase `GestorDeDescargas` operando exclusivamente contra el contrato `Descargable` .

Restricciones y Tips

- **Segregación de Capacidades:** No declare el método `descargar()` dentro de `ContenidoMultimedia` . Si lo hiciera, obligaría a `Podcast` a implementar métodos vacíos o arrojar excepciones de operación no soportada, degradando la cohesión del modelo.
- **Programación hacia Interfaces (DIP):** El `GestorDeDescargas` no debe conocer ni preguntar si un objeto encolado es una `Cancion` o una `Pelicula` (no utilice `instanceof`). Su único punto de contacto debe ser el contrato `Descargable` .
- **Seguridad Estática de Tipos:** Compruebe cómo el compilador de Java impide invocar `gestor.encolarDescarga(podcast)` con un error de tipado estático, garantizando la integridad de las operaciones en tiempo de compilación.

Preguntas Conceptuales

Al finalizar el ejercicio, reflexione sobre las siguientes preguntas:

1. ¿Por qué la capacidad `descargar()` debe modelarse en una interfaz independiente en lugar de declararse como método abstracto en la clase base `ContenidoMultimedia`?
2. ¿Qué ventajas aporta que el `GestorDeDescargas` dependa exclusivamente de la interfaz `Descargable` en lugar de acoplarse directamente a `Cancion` o `Pelicula`?
3. Si en el futuro la plataforma incorpora libros digitales (`EBook`) o manuales en PDF (que no son contenidos multimedia audiovisuales pero se pueden descargar localmente), ¿cómo se integrarían al `GestorDeDescargas` sin modificar su código?